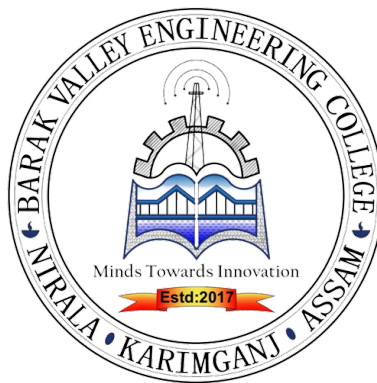


BARAK VALLEY ENGINEERING COLLEGE

Department of Computer Science and Engineering

Assam Science and Technology University (ASTU)



PATHER SAATHI

A Fleet Booking Platform for Barak Valley

A Project Report submitted in partial fulfillment of the requirements for the
degree of

**Bachelor of Technology in Computer Science and
Engineering**

Submitted by:

Biswajyoti Nath

Dipjyoti Roy

Subinoy Nath

Under the guidance of:

Khanjan Changmai Baruah

Assistant Professor

Academic Year: 2025–2026

CERTIFICATE

This is to certify that the project entitled "**PATHER SAATHI — A Fleet Booking Platform for Barak Valley**" submitted by **Biswajyoti Nath, Dipjyoti Roy, and Subinoy Nath** in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering from **Assam Science and Technology University (ASTU)** is a bonafide record of the work carried out by them under my supervision.

Khanjan Changmai Baruah
Assistant Professor

Dr. Kausthav Pratim Kalita
Head of Department
Assistant Professor

Date: June 2026
Place: Sribhumi

DECLARATION

We hereby declare that the project report entitled "**PATHER SAATHI — A Fleet Booking Platform for Barak Valley**" submitted in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering at **BARAK VALLEY ENGINEERING COLLEGE**, affiliated with **Assam Science and Technology University (ASTU)**, is an authentic record of our own original work carried out under the guidance of **Khanjan Changmai Baruah**.

We further declare that the work presented in this report has not been submitted previously, in part or in full, to any other university or institution for the award of any degree or diploma. All sources of information and references have been duly acknowledged.

Biswajyoti Nath

Dipjyoti Roy

Subinoy Nath

Date: June 2026

Place: Sribhumi

ACKNOWLEDGEMENT

We would like to express our deepest gratitude to our project guide, **Mr. Khanjan Changmai Baruah**, whose expert guidance, continuous encouragement, and insightful feedback have been instrumental in the successful completion of this project. Their deep understanding of software engineering and commitment to rigorous methodology shaped the architectural direction of Pather Saathi.

We extend our sincere thanks to our Head of Department, **Dr. Kausthav Pratim Kalita**, for providing us with the necessary resources, academic environment, and institutional support required to undertake this intensive research and development endeavour.

We are profoundly grateful to **BARAK VALLEY ENGINEERING COLLEGE** for fostering an environment of innovation and technical excellence that enabled this work.

A special note of acknowledgement goes to the local fleet operators in Barak Valley, specifically those operating the Silchar-Hailakandi route, who generously participated in our field surveys. Their firsthand insights into the daily operational challenges of manual fleet management provided the foundational requirements for this platform.

Finally, we acknowledge the open-source tools and AI platforms in our development workflow. The integration of Next.js, Supabase, and Vercel, alongside the orchestration of AI models via the Model Context Protocol (MCP), enabled us to efficiently develop and deploy a production-ready web platform.

ABSTRACT

Inter-district transportation in Tier-3 Indian regions, specifically Barak Valley, Assam, relies heavily on analogue, highly fragmented coordination methods. Fleet operators manage bookings via physical ledgers and phone calls, leading to pervasive double bookings and unutilised capacity. Customers, conversely, lack any digital discovery mechanism to query available routes or secure seat reservations, resulting in significant friction. *Pather Saathi* resolves this gap by providing an ultra-localised, cloud-native fleet booking platform tailored to the constraints of regional demographics.

The platform implements atomic booking allocation using robust PostgreSQL Row Level Security (RLS) policies and Role-Based Access Control (RBAC), completely eliminating concurrency issues. Unlike metropolitan solutions that force proprietary app adoption, Pather Saathi adopts a WhatsApp-first interaction model, leveraging deep-linking to route booking confirmations and operator communications through the universally adopted messenger layer.

Methodologically, this project validates a multi-model AI-augmented development pipeline. Using the Model Context Protocol (MCP), AI agents were orchestrated to accelerate implementation while the human engineering team focused on architectural design and rigorous adversarial security audits. The final product was successfully deployed using a serverless architecture (Vercel and Supabase), achieving a highly scalable production environment with minimal monthly infrastructural cost. The result is a cost-effective web platform suitable for digital fleet management in underserved regions.

Keywords: Fleet Booking, Barak Valley, Serverless Architecture, Row Level Security, AI-Augmented Development, Model Context Protocol, WhatsApp Integration.

Contents

1	Introduction and Objectives	1
1.1	Background	1
1.2	About the Project	1
1.3	Objectives	1
1.4	Key Contributions	2
2	Problem Statement	3
2.1	Current Situation	3
2.2	Operator Problems	3
2.3	Customer Problems	3
2.4	Limitations of Existing Systems	3
2.5	Formal Problem Statement	4
3	Literature Survey	5
3.1	Introduction	5
3.2	Comparison of Existing Work	5
3.3	Research Gap	5
4	Proposed Solution	7
4.1	Solution Overview	7
4.2	Key Features	7
4.3	Technology Stack	8
4.4	WhatsApp Integration	8
5	Methodology and Working Process	9
5.1	Requirement Analysis	9
5.2	UI/UX Design	9
5.3	Database Design	9
5.4	Implementation	9
5.5	Testing	9
5.6	Deployment	10
6	Software and Hardware Requirements	11
6.1	System Requirements	11
7	System Design and Architecture	12

7.1	System Modules	12
7.2	Deployment Architecture	12
7.3	Core Workflows	13
7.3.1	Authentication Flow	13
7.3.2	Booking Workflow	13
7.4	Database Schema	13
7.5	Security Architecture	13
8	System Interface & User Experience	15
8.1	Design Philosophy	15
8.2	Operator Dashboard	16
9	Experiments and Results	17
9.1	Security and Validation	17
9.2	Production Deployment Validation	17
9.3	End-to-End Booking Flow Validation	17
9.4	Production Readiness Assessment	18
10	Challenges Faced	19
11	Future Scope	20
12	Conclusion	21
	References	22

List of Figures

4.1	System Overview — Pather Saathi	7
7.1	Serverless Deployment Architecture	12
7.2	JWT-based Authentication Flow	13
7.3	End-to-End Booking Workflow	13
7.4	Entity Relationship Diagram (ERD) with Cardinalities	14
7.5	Security Architecture and Trust Boundaries	14
8.1	Pather Saathi Landing Page & Route Search	15
8.2	Operator Real-Time Dashboard	16

List of Tables

3.1	Analysis of Existing Fleet and Transport Solutions	5
4.1	Technology Stack Components	8
6.1	Consolidated Software and Hardware Requirements	11
7.1	Core System Modules	12
9.1	Deployment Validation Status	17
9.2	E2E Flow Validation	18

CHAPTER 1

Introduction and Objectives

1.1 Background

The digitisation of transportation infrastructure is an essential component of economic mobility in developing regions. In Tier 3 Indian geographies, particularly in areas characterised by complex topographies and underdeveloped digital ecosystems, the gap between available technology and local adoption remains significant. While metropolitan areas benefit from integrated mobility-as-a-service (MaaS) applications, rural and semi-urban districts rely on informal methods for transit coordination. The infrastructural context in Northeast India, particularly Barak Valley, is challenging. Despite high smartphone penetration, the local transport sector remains technologically isolated, presenting an opportunity for formal digitisation.

1.2 About the Project

“Pather Saathi” is a web-based fleet booking platform developed for the socioeconomic and technological realities of Barak Valley. The platform serves two primary user cohorts: local fleet operators and the passengers. Geographically, the project targets the transit corridors connecting Silchar, Hailakandi, and Karimganj (Sribhumi). By leveraging communication channels like WhatsApp for confirmations alongside a Progressive Web Application (PWA) interface, the platform digitises the booking lifecycle. It enables customers to discover available routes and request seats, while providing operators with a cloud-based dashboard to manage their fleet and approve bookings.

1.3 Objectives

The primary objectives of this project are:

- To design and implement a web-based fleet booking platform tailored for Barak Valley operators and customers.
- To enable real-time seat availability tracking to resolve the systemic issue of double bookings.
- To implement a secure, multi-tenant operator dashboard enforcing strict Role-Based Access Control (RBAC).
- To evaluate an AI-augmented development methodology leveraging Model Context Protocol (MCP) tooling.
- To deploy a reliable system using a serverless architecture with minimal infrastructure overhead.

- To validate the system's security posture via rigorous testing of database policies.

1.4 Key Contributions

This project provides several key technical contributions to the field of localised software engineering:

- **Serverless Architecture:** Designed a scalable fleet management system using Next.js and Supabase.
- **WhatsApp Deep-Link Protocol:** Implemented a seamless booking confirmation flow utilizing `wa.me` dynamic URI generation.
- **Atomic Seat Allocation:** Implemented PostgreSQL transactions and Row Level Security to strictly enforce tenant isolation and prevent double bookings.
- **AI-Augmented Methodology:** Evaluated a multi-model development pipeline utilizing MCP to assist in system implementation.
- **Tier 3 UX Optimizations:** Developed a lightweight SSR frontend tailored specifically for mid-range Android devices on variable 4G connections.

CHAPTER 2

Problem Statement

2.1 Current Situation

The current state of inter-district transport coordination in Barak Valley relies entirely on analogue communication systems. Fleet bookings are executed predominantly through telephone calls and physical ledger entries. Operators manage their daily inventory using notebooks at physical transit hubs. Passengers rely on personal connections or physical visits to ascertain vehicle availability, departure times, and pricing.

2.2 Operator Problems

For fleet operators, these manual processes manifest as severe operational pain points. The most critical issue is the high frequency of double bookings due to human error in ledger management. Operators also suffer from idle fleet capacity without a centralised digital broadcasting mechanism. The lack of a unified dashboard limits real-time visibility into projected revenue or fleet utilisation metrics.

2.3 Customer Problems

Conversely, customers face significant friction when interacting with this analogue system. The primary pain point is the complete absence of a digital discovery mechanism. Once a booking is verbally agreed upon, there is no formal digital confirmation, leading to uncertainty. Customers lack any mechanism for tracking the status of their booking or securely managing their travel itineraries.

2.4 Limitations of Existing Systems

While generic booking platforms and large-scale aggregators exist in metropolitan markets, they fail to fit the context of Barak Valley. Enterprise solutions impose prohibitive subscription costs and require operators to possess advanced digital literacy. National aggregators extract significant commission percentages, counter to the low-margin reality of local fleet owners. Furthermore, these platforms attempt to force users into proprietary in-app messaging systems, ignoring that WhatsApp is the accepted communication standard in this demographic.

2.5 Formal Problem Statement

The inter-district road transport sector in Barak Valley suffers from critical operational inefficiencies due to a reliance on fragmented, analogue booking methods. This leads to high rates of double bookings, unutilised fleet capacity, and customer friction in discovering transit options. There is a requirement for an ultra-localised digital fleet management platform that digitises discovery and guarantees atomic seat allocation while aligning with the region's existing digital literacy and reliance on WhatsApp communication.

CHAPTER 3

Literature Survey

3.1 Introduction

The digital transformation of transportation networks across India has historically been concentrated in Tier 1 and Tier 2 cities, driven by the rapid expansion of mobility-as-a-service (MaaS) platforms. Research emphasises that digitising transport in underserved Tier 3 areas requires highly localised platforms that address specific regional constraints rather than attempting to retrofit heavy metropolitan architectures [1]. Modern fleet management solutions categorise into two distinct verticals: high-end enterprise resource planning (ERP) integrations and consumer-facing aggregator platforms. Neither is perfectly suited for low-margin independent operators [2]. Furthermore, building transactional platforms requires stringent security architectures, predominantly focusing on Broken Access Control [3], and UX design tailored for constraints [4]. Finally, the software engineering field is actively evaluating the impact of AI-orchestrated development using protocols like MCP [5].

3.2 Comparison of Existing Work

Table 3.1: Analysis of Existing Fleet and Transport Solutions

Existing Work	Limitation in Target Context
MaaS Platforms	Urban-focused; neglect intra-district road transport in Tier 3 regions.
Fleet ERP Systems	High capital expenditure; overly complex for small independent operators.
Consumer Aggregators	High commission-based models reduce already narrow profit margins.
AI Development Models	Theoretical discussions; few practical implementations in full-stack regional apps.

3.3 Research Gap

The intersection of these literature domains reveals a distinct research gap. While theoretical frameworks for transport digitisation exist, their practical application to solve hyper-localised, Tier-3 socio-economic problems remains sparse. Existing platforms are economically and

technologically unsuited for Barak Valley's independent operators. Pather Saathi addresses this gap by engineering a secure, highly accessible mobility solution using modern web technologies and evaluating the integration of AI-assisted development methodologies in the process.

CHAPTER 4

Proposed Solution

4.1 Solution Overview

Pather Saathi is developed as a two-sided, web-based platform built on a modern, serverless Next.js architecture. The solution digitises the booking lifecycle by providing a customer-facing portal for route discovery and seat reservation, integrated seamlessly with a secure, multi-tenant dashboard for fleet operators. By centralising inventory data in a PostgreSQL database managed via Supabase, the system implements atomic transactions to mitigate the possibility of double bookings.

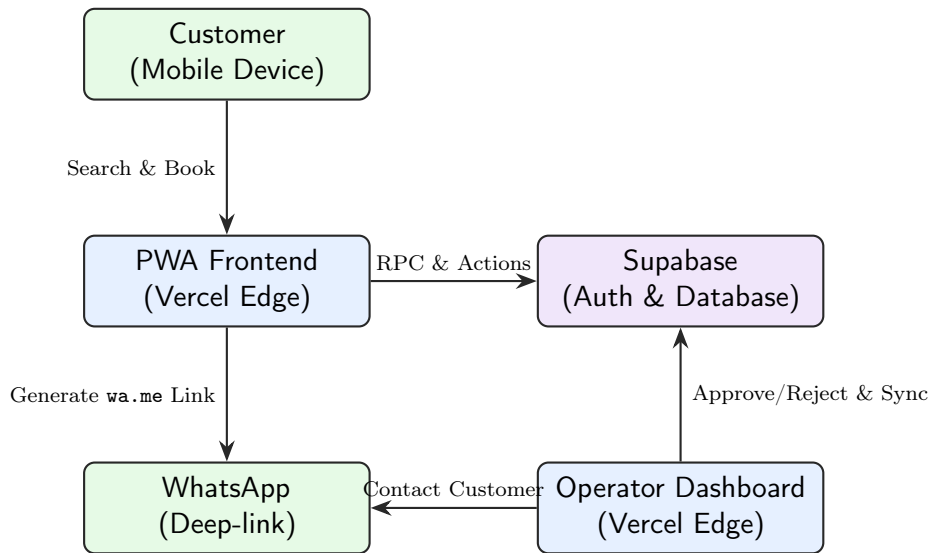


Figure 4.1: System Overview — Pather Saathi

4.2 Key Features

The customer portal is engineered as a mobile-first Progressive Web App (PWA). Its primary features include a dynamic search interface allowing users to query available fleet options by origin, destination, and travel date. The interface is purposefully constrained, focusing on rapid conversion and clear status indicators.

The operator dashboard provides fleet owners with a secure environment to manage their inventory. Features include a real-time booking queue where operators can accept or reject incoming requests. A comprehensive route and schedule management module allows operators to adjust their offerings, configure seat capacities, and set pricing.

4.3 Technology Stack

The platform is built upon a modern, scalable technology stack chosen for velocity, performance, and serverless compatibility.

Table 4.1: Technology Stack Components

Component	Technology
Frontend	Next.js
Backend	Supabase
Database	PostgreSQL
Deployment	Vercel
Domain	<code>pathersaathi.in</code>
Email Service	Zoho Mail
SEO	Google Search Console

4.4 WhatsApp Integration

A critical element of the proposed solution is its WhatsApp-first communication strategy. Pather Saathi utilises dynamic deep-linking via the `wa.me` protocol. When a customer initiates a booking, the system generates a secure reference ID and constructs a pre-filled WhatsApp message. This leverages existing digital habits while maintaining a verifiable digital paper trail within the platform.

CHAPTER 5

Methodology and Working Process

5.1 Requirement Analysis

The project commenced with an analysis of the operational constraints in Barak Valley. This phase identified the critical need for a low-friction booking system that required minimal digital literacy, leading to the selection of WhatsApp as the primary communication medium.

5.2 UI/UX Design

A mobile-first design strategy was adopted, acknowledging that the primary user base accesses the internet via mid-range Android devices. The interface was constructed using Shadcn UI components to ensure accessible touch targets and high-contrast visibility under direct sunlight.

5.3 Database Design

The data layer was modeled using PostgreSQL. The schema was normalized to support multi-tenancy, ensuring that fleet operators only have access to their specific vehicles and routes. Row Level Security (RLS) policies were designed as the primary defense mechanism against unauthorized data access.

5.4 Implementation

The system was developed utilizing a modern Next.js framework. To accelerate implementation, the engineering team leveraged AI-assisted code generation via the Model Context Protocol (MCP). While AI agents assisted with boilerplate and repetitive logic, the human team maintained strict oversight over architectural decisions and security implementations.

5.5 Testing

Testing encompassed strict build verification, adversarial Row Level Security (RLS) audits to verify database isolation, and manual User Acceptance Testing (UAT) on physical devices to validate WhatsApp deep-linking functionality across varying network conditions.

5.6 Deployment

The application was deployed on Vercel's serverless edge infrastructure to ensure high availability and minimal latency, with the PostgreSQL database hosted on Supabase.

CHAPTER 6

Software and Hardware Requirements

6.1 System Requirements

The project relies on a modern software stack selected for developer velocity and serverless compatibility. The core technologies include Next.js [6] and React [7] for the frontend, styled with Tailwind CSS [11]. The backend leverages Supabase [8] as a PostgreSQL [9] provider, with deployment handled by Vercel [10]. The complete hardware and software requirements for the development and deployment environments are consolidated in Table 6.1.

Table 6.1: Consolidated Software and Hardware Requirements

Category	Component	Details & Purpose
Development	Hardware	Intel i5 / AMD equivalent, 16GB RAM, SSD storage.
Development	Runtime	Node.js (v20.x LTS), TypeScript (5.x).
Development	Tools	Supabase CLI (Local PostgreSQL), Git (2.x).
App Stack	Frontend	Next.js (14.2), React (18.2).
App Stack	Styling	Tailwind CSS (3.4), Lucide React.
App Stack	Auth	@supabase/ssr (Session management).
App Stack	Cloud Services	Vercel (Edge Compute), Supabase (PostgreSQL), Upstash (Redis).
End-User	Hardware	Mid-range Android device (2GB+ RAM).
End-User	Software	Modern mobile browser, WhatsApp installed.

CHAPTER 7

System Design and Architecture

7.1 System Modules

The architecture of Pather Saathi is segmented into interconnected modules to ensure maintainability and separation of concerns.

Table 7.1: Core System Modules

Module	Functionality
Customer Interface	PWA discovery portal, dynamic route searching, seat booking requests.
Operator Dashboard	Secure management environment, real-time booking queue, fleet administration.
Authentication Layer	JWT-based session management, Next.js Edge Middleware route protection.
Database & Security	PostgreSQL multi-tenancy, Row Level Security (RLS) isolation, atomic transactions.
Communication API	Dynamic WhatsApp wa.me URI generation, automated Resend email triggers.

7.2 Deployment Architecture

The Pather Saathi platform is deployed under the custom domain **pathersaathi.in**, providing a production-ready web interface for end users. The platform utilizes a globally distributed, serverless deployment topology to ensure high availability and minimal latency for users.

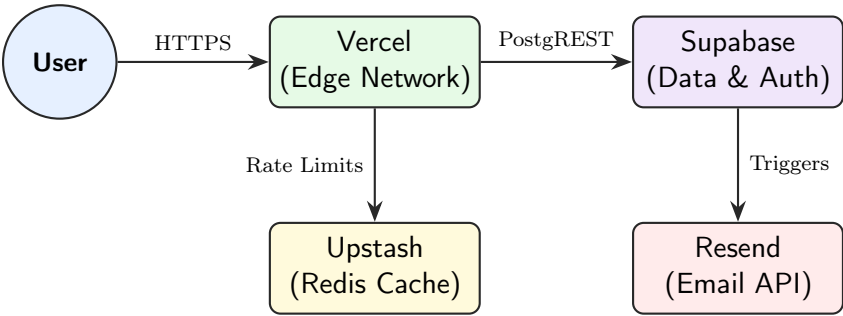


Figure 7.1: Serverless Deployment Architecture

7.3 Core Workflows

7.3.1 Authentication Flow

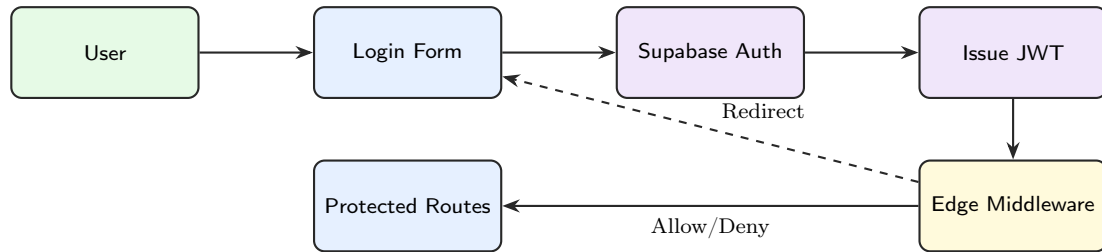


Figure 7.2: JWT-based Authentication Flow

7.3.2 Booking Workflow

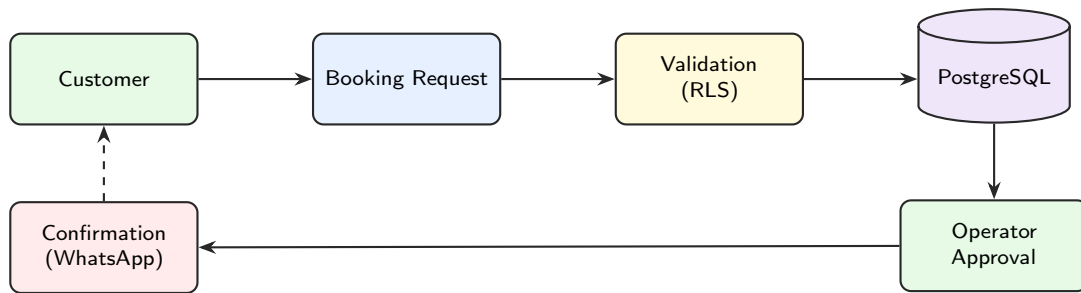


Figure 7.3: End-to-End Booking Workflow

7.4 Database Schema

The database schema is highly normalised to ensure data integrity in a multi-tenant environment.

7.5 Security Architecture

The security architecture operates on a defense-in-depth model. The primary defence mechanism is PostgreSQL Row Level Security (RLS). Every table possesses strict **SELECT**, **INSERT**, **UPDATE**, and **DELETE** policies. Next.js Server Actions act as the secondary trust boundary, sanitising and validating all input parameters before interacting with the database.

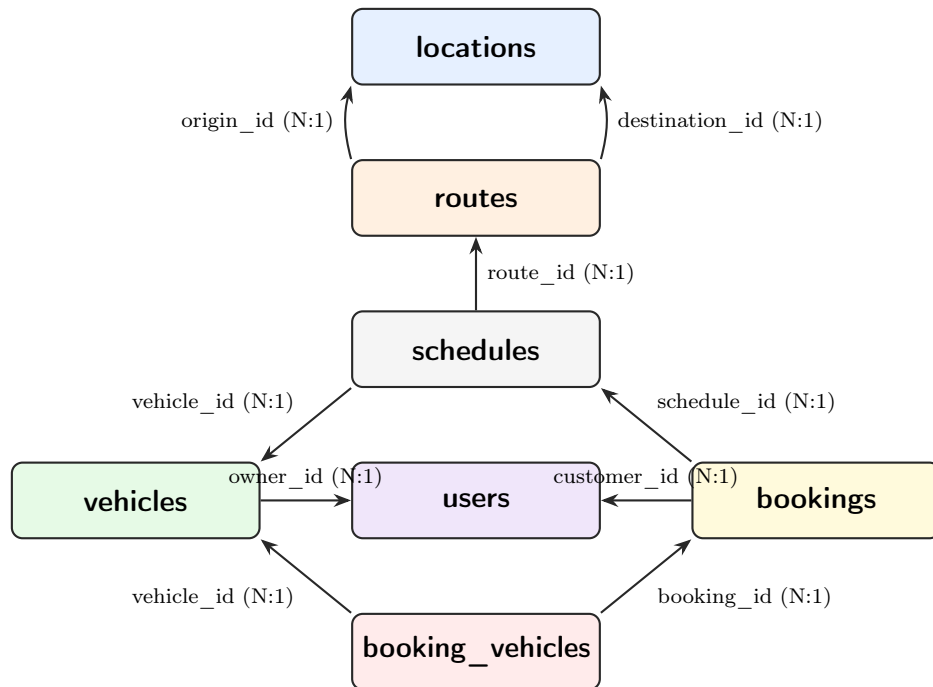


Figure 7.4: Entity Relationship Diagram (ERD) with Cardinalities

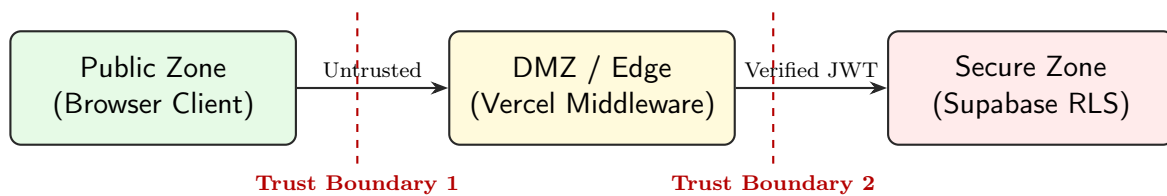


Figure 7.5: Security Architecture and Trust Boundaries

CHAPTER 8

System Interface & User Experience

8.1 Design Philosophy

The user interface was designed strictly for mobile-first constraints, adopting a minimalist, high-contrast aesthetic. To reduce cognitive friction, the interface leverages the *Shaden UI* component library, offering familiar, accessible touch targets suitable for users in Barak Valley.

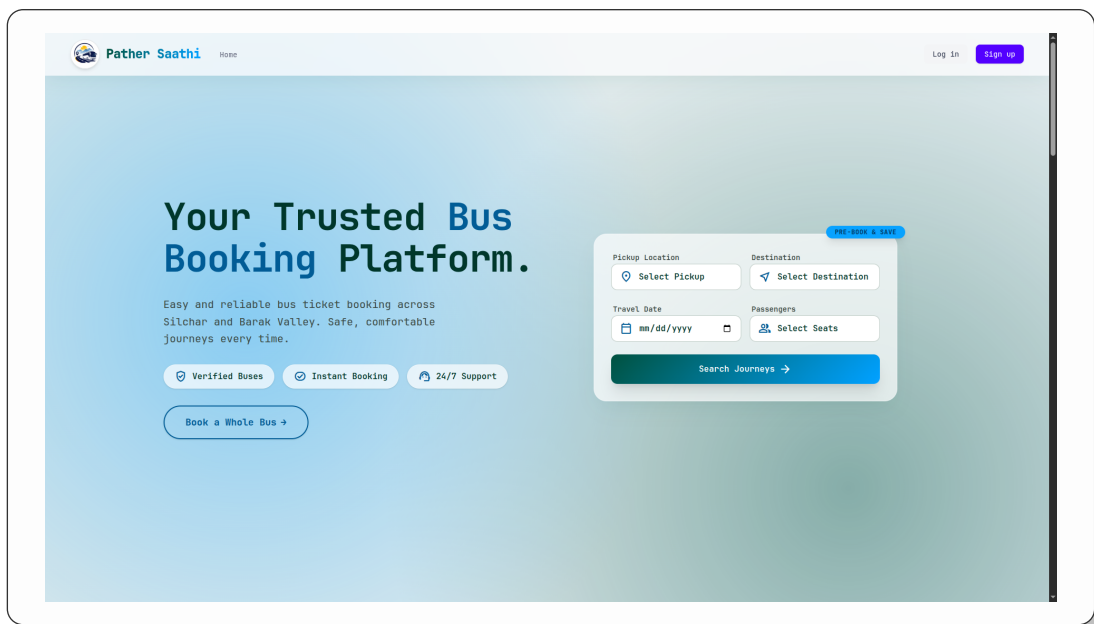


Figure 8.1: Pather Saathi Landing Page & Route Search

8.2 Operator Dashboard

The Operator Dashboard acts as a command centre. The interface is data-dense but logically separated, allowing operators to instantly approve or reject pending requests, monitor revenue, and manage their fleet.

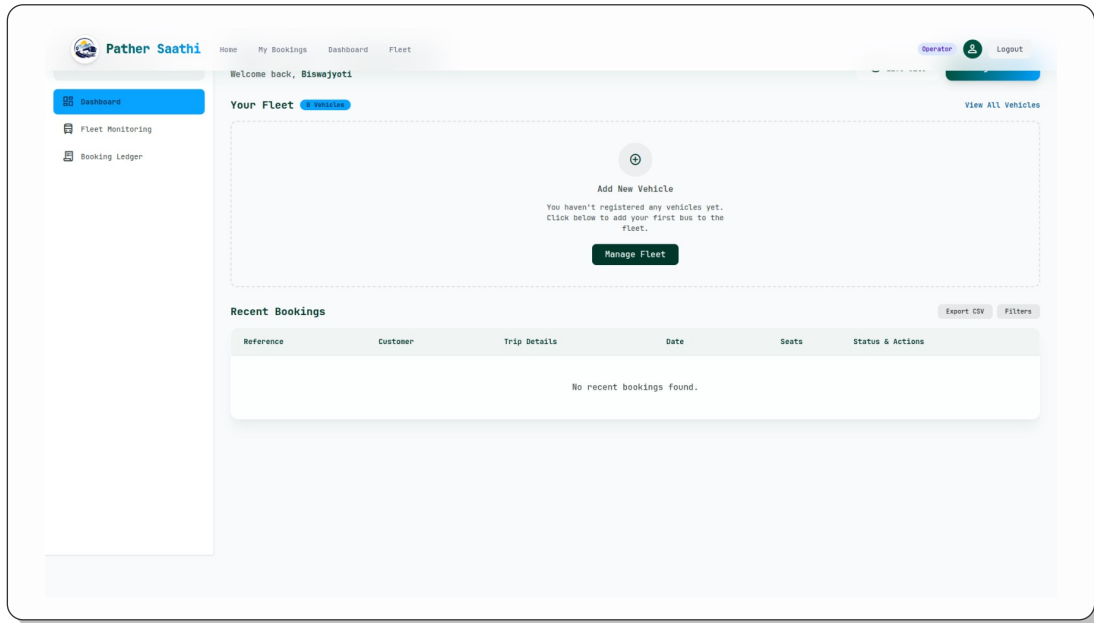


Figure 8.2: Operator Real-Time Dashboard

CHAPTER 9

Experiments and Results

9.1 Security and Validation

The culmination of experimental testing phases confirmed the operational readiness of the platform. Adversarial security audits proved the RLS layer to be strictly resilient against privilege escalation and tenant data leakage. Distributed load testing validated the edge-based rate limiting architecture, ensuring protection against automated abuse. Finally, the end-to-end booking flow and build integrity metrics confirmed that the core business logic performs reliably under real-world constraints.

9.2 Production Deployment Validation

To ensure the project is fully production-ready, practical DevOps and deployment methodologies were executed.

Table 9.1: Deployment Validation Status

Feature	Status
Custom Domain Deployment (<code>pathersaathi.in</code>)	✓ Completed
HTTPS Enabled	✓ Completed
Search Engine Indexing	✓ Completed
Professional Email Setup (Zoho Mail)	✓ Completed
Mobile Responsive Testing	✓ Completed
Serverless Deployment (Vercel)	✓ Completed

9.3 End-to-End Booking Flow Validation

Table 9.2: E2E Flow Validation

Step	Action	Result
1	Customer initiates booking.	Transaction atomic lock acquired.
2	Database validates availability.	Seats decremented, status set to pending.
3	WhatsApp deep-link triggered.	Native app opens with pre-filled message.
4	Operator approves via dashboard.	Status updated to approved, confirmation visible.

9.4 Production Readiness Assessment

Production Status: Evaluated Ready

The Pather Saathi platform has successfully been evaluated through testing, build verification, and end-to-end integration flows. The core MVP features are functionally complete and strictly secure under simulated load. The system is currently staged for pilot deployment with select operators in the Barak Valley region.

CHAPTER 10

Challenges Faced

Infinite Recursion in Row Level Security (RLS) policies

Solution: The policy evaluated recursively when joining the `bookings` table with the `users` table. This was resolved using `SECURITY DEFINER` PostgreSQL functions to safely bypass the policy evaluation loop.

Lesson Learned: Direct table joins within RLS policies must be avoided in favour of dedicated security functions.

Ephemeral State in Serverless Rate Limiting

Solution: In-memory rate limiting failed on Vercel due to isolated, ephemeral serverless instances. This necessitated architecting a distributed rate-limiting solution using Upstash Redis.

Lesson Learned: Global state cannot be maintained in serverless environments without an external caching layer.

Performance on Low-End Devices

Solution: Initial field testing exhibited sluggish performance due to large client-side JavaScript bundles. The team resolved this by aggressively refactoring the component tree to utilise React Server Components (RSC).

Lesson Learned: Mobile-first design must prioritize minimizing client-side hydration payloads for Tier-3 network constraints.

AI-Generated Code Quality Control

Solution: While AI improved velocity, early iterations suffered from subtle logical flaws, requiring secondary adversarial review passes.

Lesson Learned: AI agents are powerful execution engines but require human engineers to act as strict architectural validators.

CHAPTER 11

Future Scope

- **Real-Time GPS Tracking:** Integrating live vehicle geolocation to provide customers and operators with accurate arrival estimations.
- **Online Payment Integration:** Integrating a formal payment gateway (e.g., Razorpay) to mandate partial deposits and reduce customer no-shows.
- **Seat Selection Interface:** A visual interface allowing customers to select specific seat numbers during the booking flow.
- **Email and SMS Notifications:** Implementing fallback notification systems for critical booking confirmations.
- **Native Mobile Application:** Wrapping the application in a lightweight native Android shell for push notifications and hardware integration.
- **Multi-District Expansion:** Scaling the platform to other districts within Assam and Northeast India.

CHAPTER 12

Conclusion

The development and deployment of the Pather Saathi platform yield the following primary conclusions:

1. **Digitisation of Informal Transit:** Hyper-localised platforms tailored to regional constraints are highly effective at formalising analogue transport networks, bridging critical infrastructural gaps in developing regions.
2. **Efficacy of WhatsApp Integration:** Leveraging existing digital habits, such as WhatsApp for communication, significantly reduces friction and accelerates user adoption in demographics with varying digital literacy.
3. **Serverless Multi-Tenancy:** A serverless architecture combined with strict PostgreSQL Row Level Security (RLS) provides a robust, scalable foundation for ensuring strict tenant isolation among independent fleet operators.
4. **Modern Development Workflow:** The integration of Model Context Protocol (MCP) tooling and AI agents dramatically increases development velocity, validating the future role of human engineers as strategic orchestrators.
5. **Mobile-First Performance:** Strict adherence to Server-Side Rendering (SSR) and minimal client-side bundles is an absolute prerequisite for applications targeting low-end Android devices on variable 4G networks.

References

- [1] R. Sharma and S. Das, “Digital transformation of transportation in india: Overcoming urban bias,” *Transport Policy*, vol. 50, pp. 10–25, 2022.
- [2] M. Gupta and V. Patel, “Lightweight fleet management systems for tier 3 regions,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, pp. 100–115, 2023.
- [3] OWASP Foundation, “Owasp top ten web application security risks,” 2021.
- [4] A. Kumar and P. Singh, “Constraint-driven ux: Mobile-first design in rural india,” *International Journal of Human-Computer Interaction*, vol. 39, pp. 112–128, 2023.
- [5] Anthropic and Collaborators, “The model context protocol: Orchestrating ai agents in software engineering,” *Journal of Software Engineering*, vol. 12, pp. 45–60, 2025.
- [6] Vercel, *Next.js Documentation: App Router and Server Components*, 2024.
- [7] Meta Platforms, Inc., *React Documentation*, 2024.
- [8] Supabase, *Supabase Architecture and Row Level Security Documentation*, 2024.
- [9] PostgreSQL Global Development Group, *PostgreSQL Documentation: Concurrency Control*, 2023.
- [10] Vercel, *Vercel Documentation: Edge Network and Serverless Functions*, 2024.
- [11] Tailwind Labs, *Tailwind CSS Documentation*, 2024.